Universidad Autónoma del Estado de México

FACULTAD DE INGENIERÍA

PLAN DE ESTUDIOS:
Licenciatura de Ingeniería en Computación

UNIDAD DE APRENDIZAJE:
Robótica

Tarea 5

“DETECCIÓN DE OBJETOS CON ESP32-CAM Y EDGE IMPULSE”

INTEGRANTES DE EQUIPO:

- Fernando Azuara Ibarra

GRUPO:
O1

DOCENTE:
DR. EN C.I.E. JAIME GARCÍA GARCÍA

Toluca, Estado de México, 10 de noviembre de 2026

1. INTRODUCCIÓN

El aprendizaje automático en microcontroladores (**TinyML**) permite ejecutar un modelo directamente en un dispositivo de bajo consumo, lo que reduce la latencia, evita enviar imágenes a internet y permite operar sin conexión [4], [5]. **Edge Impulse** integra en una sola plataforma la captura de datos, el entrenamiento, la cuantización y la generación de código [2], y su familia de modelos **FOMO** (Faster Objects, More Objects) entrega el centroide de cada objeto en lugar del cuadro completo, a cambio de correr más rápido en hardware limitado [3], [11]. En esta práctica se entrenó un detector de tres objetos propios con la cámara de un **ESP32-CAM** (AI-Thinker, sensor OV3660), se desplegó como librería de Arduino y se midió su desempeño con 45 pruebas, siguiendo el video base [1]. Un vaso y una jarra transparentes con diseño rojo se eligieron a propósito por parecerse; el tercer objeto es una laptop negra. El objetivo es obtener un detector que funcione **sin internet** y analizar en qué condiciones acierta y en cuáles falla.

2. METODOLOGÍA

2.1 Captura del dataset en condiciones controladas

Las fotos se tomaron con la cámara del propio robot (QVGA, 320×240, JPEG) por WiFi, desde una página para el celular servida por un script en la PC, en lugar de la librería Eloquent del video. La cámara quedó **fija a 10 cm de altura**, con una mesa plegable blanca como fondo a 30 cm detrás del objeto, un foco del cuarto contiguo como única luz y **sin flash**. Los objetos se colocaron en un punto marcado, a 55 cm de la cámara el vaso y la jarra y a 75 cm la laptop, que a 55–60 cm se salía del encuadre. El objeto se giró en cada toma y se verificó que quedara completo dentro del recorte central de 240×240 px que usa el modelo (Fit shortest axis, líneas amarillas de la Figura 1); las fotos con el objeto cortado se apartaron (37 en la primera captura). La Tabla 1 resume el dataset final.

Objeto	Fotos	Distancia	Entren./Prueba	Particularidad
vasorojo	60	55 cm	48 / 12	Vaso transparente con diseño rojo
jarraroja	60	55 cm	47 / 13	Jarra transparente con diseño rojo; oreja con cinta amarilla, a ambos lados y de tres cuartos
laptop	60	75 cm	48 / 12	Negra, abierta unos 90°, girada unos 45° siempre al mismo lado
Total	180	—	143 / 37	Reparto automático 80 % / 20 % de Edge Impulse

Tabla 1. Dataset final (versión 3) y condiciones de captura.



Figura 1. Muestra de cada clase con el recorte central del modelo (líneas amarillas).



Figura 2. Montaje físico (cámara a 10 cm, fondo blanco), página de captura en el celular y página de las 45 pruebas.

2.2 Etiquetado y entrenamiento en Edge Impulse

Las imágenes se subieron con la CLI de Edge Impulse al proyecto público [enlace en la sección 5] y se etiquetaron con **un cuadro ajustado por imagen** (el de la jarra incluye la oreja). El impulse recorta al centro y reduce a **96×96 px**, usa el bloque **FOMO MobileNetV2 0.35** y se entrenó con tasa de aprendizaje 0.001, aumento de datos y cuantización int8. Se usó **RGB** y no escala de grises (justificación en la sección 4). El proceso fue iterativo (Tabla 2): con el dataset inicial la laptop se confundía con el fondo (F1 de 0.67); subir los ciclos de 60 a 100 no ayudó, y el diagnóstico apuntó a la captura (laptop demasiado grande y 12 cuadros fuera del recorte), así que se **recapturó la laptop a 75 cm**. Ya desplegado, la jarra se confundía con el vaso, por lo que se puso **cinta amarilla en la oreja**, se recapturaron las 60 fotos de la jarra, se reetiquetaron y se reentrenó. La Figura 3 muestra la matriz de confusión antes y después.

Iteración	Cambio	Ciclos	F1 validación	Prueba real (45)
1	Dataset v1 (laptop a 55 cm)	60	90.5 % (laptop 0.67)	—
2	Ajuste de un parámetro: más ciclos	100	90.0 % (laptop 0.60)	—
3	Dataset v2: laptop recapturada a 75 cm	60 y 100	100 %	39/45 (86.7 %), con flash
4	Dataset v3: cinta amarilla en la oreja de la jarra	60 y 100	100 %	40/45 (88.9 %), sin flash

Tabla 2. Evolución del modelo. La validación de Edge Impulse usa solo 23 objetos, por lo que no basta para comparar.

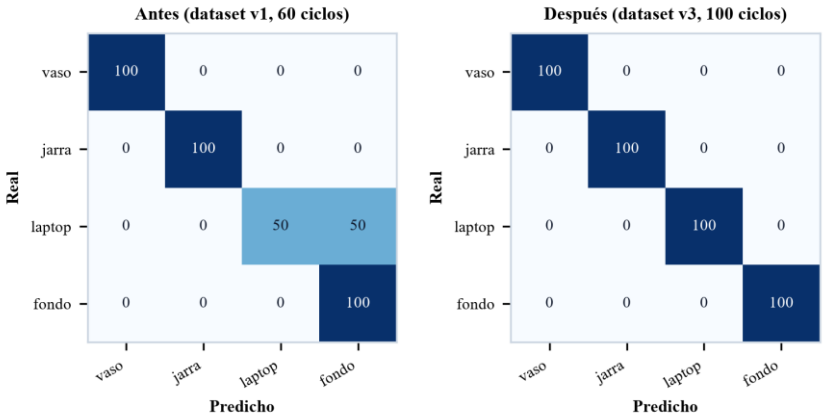


Figura 3. Matriz de confusión de validación (%) antes y después del ajuste.

2.3 Despliegue en el ESP32-CAM

Se exportó la librería de Arduino (EON Compiler, int8) y se instaló como .ZIP. El sketch deteccion_ei.ino captura QVGA con el mismo volteo vertical que el robot, decodifica el JPEG a RGB888, recorta al centro, reduce a 96×96 y ejecuta el clasificador. Imprime por serie una línea por cuadro con **etiqueta, probabilidad y coordenadas (x, y)** del centroide (DET;t;dsp;clasif;etiqueta;prob;cx;cy;w;h) o NONE. Compila con el core ESP32 3.3.12 (el video usa 3.0.7) y ocupa 14 % de la flash y 12 % de la RAM. Hubo dos problemas: (a) en este core fmt2rgb888 entrega los bytes en orden R, G, B y el ejemplo de Edge Impulse asume B, G, R, de modo que el vaso rojo llegaba al modelo como azul y no se detectaba nada; se corrigió y se verificó volcando por serie la imagen que ve el modelo; (b) el destello del LED al detectar iluminaba el cuadro siguiente (el dataset no usó flash) y calentaba el LED, por lo que se desactivó. Al detectar, el robot **hace un gesto**: si la misma clase domina en 2 de 3 cuadros, el vaso hace que asienta (adelante y atrás), la jarra que niega (giros izquierda, derecha, izquierda) y la laptop queda una vuelta completa (1.74 s, el tiempo de 360° calibrado en la Tarea 4 con PWM 140, unos 365°), con PWM limitado y una pausa de 3 s para no repetirlo. Una página para el celular, servida por el propio ESP32, muestra la foto, la clase detectada y el gesto en curso. **No se implementó el reto de la pantalla OLED** (extra opcional) porque el ESP32 sirve su propia página web: el celular se conecta a su red WiFi (ESP32Det_Fer) y muestra la foto del cuadro, la clase, la probabilidad, el gesto en curso, el contador y el registro de detecciones. Esa pantalla es más grande y legible que una OLED de 0.96", no ocupa los pines I2C (GPIO14 y GPIO15 ya mueven los motores en este robot) y funciona sin computadora. **[Pendiente: describir la prueba autónoma con power bank y anexar fotos o video.]**

2.4 Pruebas

Con el robot conectado por USB, un script lee las líneas seriales y se hicieron **45 pruebas**: por objeto, 10 en la posición del entrenamiento (movimientos mínimos) y 5 en otra posición o con otra luz (más cerca, pegado al fondo, con linterna o lámpara). Cada prueba toma una ventana de 3 s (unos 3 cuadros) y registra la detección de mayor probabilidad; cuenta como acierto si coincide con el objeto. Se calcularon intervalos de Wilson al 95 % y pruebas exactas de Fisher.

3. RESULTADOS

El modelo final acertó **40 de 45 pruebas (88.9 %, IC95 76.5–95.2 %)**. En la posición del entrenamiento acertó **30 de 30**, y en otra posición o luz **10 de 15** (Fisher, p = 0.002). La laptop acertó 15 de 15. En la jarra, el ajuste con cinta subió el acierto en la posición del entrenamiento de 6/10 a 10/10 (Tabla 3, tanda 1 = modelo sin cinta y con flash). La tabla completa de las 45 pruebas (objeto, prueba, etiqueta, probabilidad, coordenadas y acierto) está en el anexo.

Objeto	Posición entren.	Otra pos./luz	Total	IC95	Tanda 1
vasorojo	10/10	3/5	13/15	62–96 %	15/15
jarraroja	10/10	2/5	12/15	55–93 %	9/15
laptop	10/10	5/5	15/15	80–100 %	15/15
Global	30/30	10/15	40/45	77–95 %	39/45

Tabla 3. Aciertos por objeto y condición (tanda 1: modelo sin cinta, con flash).

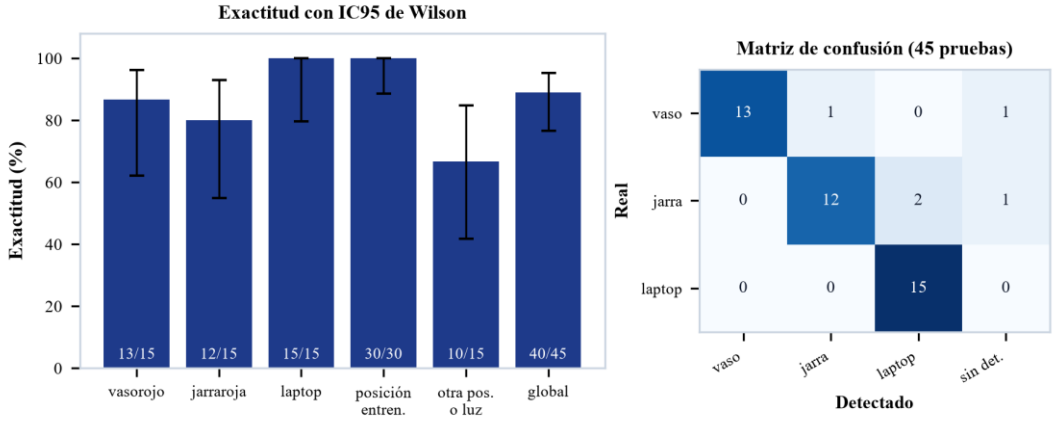


Figura 4. Exactitud por grupo y matriz de confusión de las 45 pruebas reales.

Prueba	Condición	Detectado	Prob.
vasorojo 1	Se puso pegado a el fondo	sin detección	—
vasorojo 4	Se acerco mucho a la cámara	jarraroja	0.75
jarraroja 3	Se acercó de más	sin detección	—
jarraroja 4	Se iluminó con linterna, oreja del lado derecho	laptop	0.60
jarraroja 5	Se iluminó con lampara, oreja lado izquierdo	laptop	0.77

Tabla 4. Las pruebas fallidas (todas de la 11 a la 15 de cada objeto).

En el ESP32 cada cuadro tarda **798 ms** (26 ms de procesado y 772 ms de clasificación), unos **1.25 cuadros por segundo**, frente a los 628 ms que estima Edge Impulse para un Cortex-M4F; el modelo usa 133 KB de RAM y 81 KB de flash según el estimador. La probabilidad de los aciertos (0.83 ± 0.13) fue algo mayor que la de los errores (0.71 ± 0.09), sin diferencia significativa (Mann-Whitney, $p = 0.11$), por lo que la probabilidad sola no sirve como filtro. Con $n = 15$ por objeto los intervalos son amplios.

4. ANÁLISIS Y CONCLUSIONES

¿Por qué cámara fija, fondo constante y mismo punto? Con unas 60 fotos por clase el modelo aprende todo lo que se repite en las imágenes, incluido el escenario; fijar la cámara, el fondo y la luz reduce la variación que no importa. Al cambiar la posición o la luz la exactitud bajó de 100 % (30/30) a 66.7 % (10/15): fallaron el objeto pegado al fondo, muy cerca de la cámara y con luz fuerte.

¿Qué se confundió y qué se hizo? Vaso y jarra, por tener el mismo color y diseño transparente: en la primera tanda 11 de las 15 pruebas de la jarra mostraron una etiqueta falsa de vaso y solo 9 acertaron. Subir ciclos no mejoró nada; lo que sí funcionó fue **añadir una marca distintiva (cinta amarilla en la oreja)** y reentrenar. Esa mejora viene de un marcador añadido y no de un modelo mejor, y el modelo puede apoyarse en el color de la cinta. La laptop mejoró al recapturarla a 75 cm. Con luz fuerte la jarra se detectó como laptop.

¿Por qué escala de grises y no RGB? Un cuadro en grises tiene un tercio de los datos de entrada (9,216 contra 27,648 valores), lo que reduce memoria y tiempo en un ESP32, y por eso lo recomienda el enunciado. En esta práctica se usó **RGB** de forma deliberada: (a) los objetos se distinguen sobre todo por el **color** (vaso y jarra son rojos, la laptop es negra y la marca que separa vaso de jarra es una cinta amarilla), y al pasar a grises el rojo y el amarillo se convierten en tonos parecidos y esa información se pierde; (b) los recursos alcanzan con RGB, pues el modelo usa 14 % de la flash, 133 KB de RAM estimados y 772 ms por clasificación, sin llegar a ningún límite del ESP32-CAM; y (c) el aumento de tamaño del modelo con RGB afecta solo a la primera capa. No se midió una corrida en grises, así que el ahorro y la posible pérdida de exactitud no están cuantificados y quedan como trabajo futuro.

Ventajas de inferencia local. Menor latencia, porque no hay viaje de ida y vuelta a un servidor; privacidad, porque las imágenes no salen del dispositivo; funcionamiento sin internet; y menor costo y consumo. A cambio, el modelo debe caber en la memoria y el cómputo del microcontrolador (1.25 cuadros por segundo en este caso).

Aplicación industrial con (x, y). Un clasificador de residuos: las coordenadas del centroide se convierten, con una calibración de la cámara fija sobre la banda, a posición en el plano de trabajo, y un brazo robótico toma la pieza con cinemática inversa y la deposita en el contenedor de su clase.

Centroides de FOMO frente a cuadros delimitadores. El centroide basta para el rumbo (el error de dirección es la distancia del centro a la mitad de la imagen), pero FOMO trabaja en celdas de 8×8 px sobre 96×96, tiene resolución gruesa y no da el tamaño real del objeto; un cuadro permite estimar la distancia por su ancho y decidir cuándo detenerse.

Conclusiones. Se logró un detector que corre completo en el ESP32-CAM, sin internet, con **88.9 %** de acierto en 45 pruebas y 100 % en la posición del entrenamiento. Su límite es la generalización: falla con cambios fuertes de distancia, fondo y luz, y distingue vaso de jarra solo gracias a un marcador. Para mejorarlo harían falta más fotos con esas variaciones, una evaluación con más pruebas y la comparación RGB contra grises.

Enlaces. Repositorio: https://github.com/igream/ESP32-AI-Robot · Proyecto de Edge Impulse: https://studio.edgeimpulse.com/public/1159326/live

REFERENCIAS

[1] The Professor Garcia, "Entrenando IA desde cero para reconocer objetos nuevos con ESP32-CAM, Edge Impulse, Arduino," YouTube, 11 dic. 2024. [Online]. Available: https://www.youtube.com/watch?v=9uhKfEK73nA. Accessed: Nov. 8, 2026.

[2] Edge Impulse, "Announcing Official Support for the Espressif ESP-EYE (ESP32)," 27 abr. 2022. [Online]. Available: https://www.edgeimpulse.com/blog/announcing-official-support-for-the-espressif-esp-eye-esp32/. Accessed: Nov. 8, 2026.

[3] Edge Impulse, "Add Sight to Your ESP32," 8 sep. 2021. [Online]. Available: https://www.edgeimpulse.com/blog/add-sight-to-your-esp32/. Accessed: Nov. 8, 2026.

[4] R. Kallimani, K. Pai, P. Raghuwanshi, S. Iyer and O. L. A. López, "TinyML: Tools, applications, challenges, and future research directions," Multimedia Tools and Applications, vol. 83, no. 10, pp. 29015–29045, 2024 (publicado en línea sep. 2023), doi: 10.1007/s11042-023-16740-9. [Online]. Available: https://doi.org/10.1007/s11042-023-16740-9

[5] P. P. Ray, "A review on TinyML: State-of-the-art and prospects," J. King Saud Univ. – Comput. Inf. Sci., vol. 34, no. 4, pp. 1595–1623, abr. 2022, doi: 10.1016/j.jksuci.2021.11.019. [Online]. Available: https://doi.org/10.1016/j.jksuci.2021.11.019

[11] N. Badalyan, "FOMO Edge Impulse detecte potato beetle," Hackster.io, 23 jun. 2022. [Online]. Available: https://www.hackster.io/norik-badalyan/fomo-edge-impulse-detecte-potato-beetle-6ac2a7. Accessed: Nov. 8, 2026.